

第二章 可行性研究与计划 AI讲解

开篇导读

一句话概括：可行性研究是“用最小的代价、在最短的时间内，确定问题是否值得去解决”——它不是真的解决问题，而是判断“这个问题值不值得花大价钱去解决”。

本章在考试中的地位：中等频率考点章。高频考点是数据流图（DFD）的符号与分层、三类可行性的场景判断、数据字典与DFD的关系。本章的DFD和数据字典还会在第三章需求分析中复用，是结构化分析方法的基石。

一、可行性研究的实质

核心概念

可行性研究不是去解决问题本身，而是用最小的代价、在尽可能短的时间内，确定问题是否值得去解决。

为什么重要

理解“实质”是解题的关键。可行性研究的实质是：一次简化、压缩了的系统分析和设计过程。也就是说，它不是“要不要做”的拍脑袋决定，而是用一个“迷你版”的分析设计来评估可行性。它要回答的不是“怎么做”，而是“做不做”。

易混辨析

考法	易错点
"可行性研究的实质"	是"简化压缩的系统分析与设计"，不是"详细的需求分析"。它比需求分析更早、更粗略
"可行性研究的目的"	是"确定问题是否值得解决"，不是"解决问题"或"确定怎么解决"

记忆技巧

口诀：“小代价、短时间、定值不值”——可行性研究就是“花小钱办小事，先看看值不值得花大钱”。

可行性研究的8个步骤（简答题高频考点）

可行性研究不是拍脑袋，而是一套有步骤的迷你分析流程：

1. 复查系统规模和目标：确认要解决的问题是什么、边界在哪
2. 研究目前正在使用的系统：现有系统怎么做的、有什么问题（现有系统是新系统的参考）
3. 导出新系统的高层逻辑模型：画系统级DFD，粗略描述新系统数据流
4. 重新定义问题：基于逻辑模型，再次确认问题定义是否准确
5. 导出和评价供选择的解法：想出多个可行方案，逐一评价
6. 推荐行动方针：从候选方案中选最优，推荐做什么

7. 草拟开发计划：粗略排进度、估成本、定人员

8. 书写文档提交审查：把以上内容写成可行性研究报告

恍然大悟：为什么是8步而不是直接判断？因为"判断值不值得做"本身就是一个迷你版的开发流程——先搞清楚问题（1-4步），再想方案（5-6步），最后排计划（7-8步）。这就是"简化压缩的系统分析设计"的含义：它不是跳过分析直接拍板，而是用最小成本走一遍分析设计的核心流程，再决定要不要正式开发。

二、三类可行性（必背）

核心概念

可行性研究从三个维度评估（部分教材加第四个维度）：

可行性	评估什么	通俗解释
技术可行性	现有技术能否实现	"做得出吗？"
经济可行性	成本效益是否划算	"做得起吗？值吗？"
操作可行性	运行环境是否可行、用户能否接受	"用得起来吗？"
（法律/社会可行性）	是否违法、社会影响	"合法吗？"

为什么重要

这是选择题高频考点，出题方式是给定场景判断属于哪类可行性。设坑点是"技术"和"操作"的混淆。

三类可行性深度拆解

1. 技术可行性

- 核心问题：现有技术能否实现该系统？
- 评估四子维度：

1. 硬件能力：服务器、网络、存储是否够用（如要支撑10万并发，单机CPU/内存够吗）

2. 软件平台：操作系统、数据库、中间件是否有现成方案

3. 技术成熟度：所需技术是否成熟稳定（如用前沿AI模型 vs 用成熟SQL）

4. 团队技术水平：开发人员能否掌握所需技术

- 输出：技术风险等级（高/中/低）+ 关键技术验证方案
- 典型结论：要么"技术可行（已有成熟方案）"，要么"需先做技术预研"，要么"技术不可行（需更换方案）"

2. 经济可行性

- 核心问题：项目在经济上是否值得做？
- 评估四子维度：

1. 开发成本：人力、设备、第三方服务费等一次性投入

2. 运维成本：上线后的服务器、维护人力、升级费等持续投入

3. 收益预测：直接收益（增加销售）+ 间接收益（提升效率、改善服务）

4. 投资回收期：多久能回本（一般要求≤3年）

- 核心方法：成本效益分析（含货币时间价值，详见本章第六节）

- 典型结论：算出"纯收入"、"投资回收期"、"投资回报率"，与企业要求对比

3. 操作可行性

- 核心问题：系统建好后能否真正运行起来？
- 评估三子维度：

1. 用户接受度：用户愿不愿用（用户习惯、培训成本、抵触情绪）
2. 组织变革阻力：是否冲击现有岗位/利益（如自动化裁掉某岗位）
3. 运行环境匹配：实际工作环境（网络、设备、安全要求）能否承载

- 典型反例：某医院花500万开发了HIS系统（技术、经济都可行），但医生坚持用纸质病历，系统没人用——操作不可行
- 典型结论：评估变革风险 + 制定培训和推广计划

易混辨析（重点）

场景	属于哪类	易错分析
"现有服务器能否支撑10万并发"	技术可行性	涉及硬件能力、算法实现 → 技术
"系统上线后员工能否熟练操作"	操作可行性	涉及人的使用、运行环境 → 操作
"开发成本50万，5年收益200万"	经济可行性	涉及钱、成本、收益 → 经济
"系统是否符合数据保护法"	法律可行性	涉及法规 → 法律

技术 vs 操作的区分关键：

- 技术可行性问的是"技术上能不能实现"（硬件、软件、算法、人员技术水平）
- 操作可行性问的是"实现后能不能用起来"（运行环境、用户接受度、组织流程）

恍然大悟：为什么是这3个维度而非其他？因为三类可行性形成了漏斗式评估顺序——先问"能不能做（技术）"→再问"值不值得做（经济）"→最后问"用不用得起来（操作）"。三关缺一不可：技术不可行直接死掉；技术可行但经济不划算就放弃；技术经济都OK但用户不买账就成"摆设系统"。很多失败项目栽在第三关——技术经济都过了，没考虑组织变革阻力，建好系统没人用。技经操不是并列三选一，而是逐层筛选的过滤器。

记忆技巧

口诀："技经操（机灵操）"——技术、经济、操作。谐音"机灵操作"：先看技术能不能做（机），再看经济划不划算（灵），最后看操作能不能用起来（操）。

三、系统流程图

核心概念

系统流程图是描述物理系统的传统工具，它用图形符号表示系统中各个物理元素（程序、数据、人工处理等）以及数据在这些元素之间流动的情况。

为什么重要

系统流程图描述的是"物理系统"（实际怎么运行），而后面要讲的DFD描述的是"逻辑系统"（数据怎么流动）。两者层次不同，容易混淆。

易混辨析

工具	描述什么	阶段
----	------	----

系统流程图	物理系统（实际运行方式）	可行性研究
数据流图（DFD）	逻辑系统（数据流动）	可行性研究 + 需求分析

系统流程图的常用符号

系统流程图描述物理元素（程序、数据、人工处理等）及数据流动：

符号形状	表示什么	通俗解释
矩形	处理/程序	一个程序或处理步骤
平行四边形	数据	输入/输出的数据
波浪底矩形	文档	打印出的纸质文档
圆柱	联机存储	数据库/磁盘存储
菱形	判断	条件判断（如“是否有效？”）

简单例子：工资处理系统的物理流程

[考勤数据] → (核对考勤) → {考勤表} → (计算工资) → [工资单] → (打印) → 工资条

• 考勤数据（平行四边形）→ 核对考勤程序（矩形）→ 存考勤表（圆柱）→ 计算工资程序（矩形）→ 工资单（平行四边形）→ 打印（矩形）→ 工资条文档（波浪底矩形）
恍然大悟：系统流程图为什么描述“物理”？因为它画的是“实际怎么跑”——哪个程序处理什么数据、存到哪个磁盘、打印出什么文档。而DFD画的是“逻辑”——数据从哪流到哪，不管实际用什么程序跑。系统流程图偏“实”（物理实现），DFD偏“虚”（逻辑功能）。

优缺点与适用边界

- 优点：直观展现现有物理系统的实际运行方式（哪个机器、哪份纸、哪个程序），便于和业务人员沟通现状
- 缺点：
 1. 过早绑定技术方案——画完图就锁定“用什么数据库、用什么打印机”，影响后续系统设计的灵活性
 2. 抽象层次低——一旦物理实现变化（如纸质改电子），整张图都要重画
 3. 不适合描述新系统的逻辑功能——只能照搬现状，无法表达“应该怎么做”的需求
- 适用阶段：仅在可行性研究阶段用来理解和描述现有系统，不用于新系统设计

系统流程图 vs DFD：何时用哪个

决策依据	用系统流程图	用DFD
描述对象	现有系统（怎么跑）	新系统（要做什么）
抽象层次	物理层（具体设备/程序/文档）	逻辑层（数据流动）
关注点	实现细节	功能本质
阶段	可行性研究（理解现状）	可行性研究 + 需求分析（设计新系统）

实际工作中两者配合：先画系统流程图理解客户现有的人工/旧系统流程 → 抽离出数据流动的逻辑 → 画DFD描述新系统应该怎么做。系统流程图是“现状照片”，DFD是“未来蓝图”。

恍然大悟：系统流程图的局限不是“画错了”，而是“问错了问题”——它问的是“现在怎么跑”，但软件工程真正要回答的是“未来该怎么做”。所以它在可行性研究的角色定位是“调研工具”而非“设计工具”，画完就该被DFD取代。这也解释了为什么很多教材把系统流程图放在“可行性研究”章而把DFD放在“需求分析”章——前者帮你理解现状（输入），后者帮你设计未来（输出）。

记忆技巧

口诀：“系统流程图描述物理，数据流图描述逻辑”——系统流程图偏“实”（物理），DFD偏“虚”（逻辑）。

四、数据流图 (DFD) 核心考点

核心概念

数据流图（DFD，Data Flow Diagram）是一种图形化技术，它描绘信息流和数据从输入移动到输出的过程中所经受的变换。

四种基本符号（必背图形对应）

符号	形状	含义
源点/终点	正方形（或立方体）	数据的来源或去向（外部实体）
处理/变换	圆角矩形（或圆形/圆圈）	对数据进行变换/处理
数据存储	开口矩形（或两条平行横线）	数据暂存的地方
数据流	箭头	特定数据的流动方向

分层（高频考点）

DFD采用自顶向下逐层分解的方式:

- 顶层图：把整个系统看作一个处理，显示系统与外部实体的交互
- 0层图（0层）：把顶层图的处理分解为若干子处理
- 1层图、2层图...：继续逐层细化

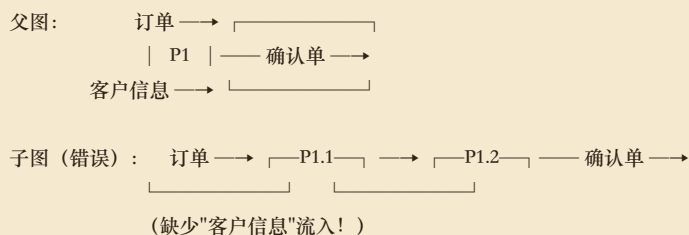
父图与子图的平衡原则（必考）

平衡原则：子图的输入输出数据流，必须与父图中对应处理的输入输出数据流一致。

这是DFD分层最核心的规则。父图中一个处理被分解成子图后，子图所有外部数据流的"总和"要等于父图中那个处理的数据流。如果不平衡，说明分解有误。

不平衡反例（常见错误）：

父图中处理P1有2条流入数据流（订单、客户信息）和1条流出数据流（确认单），但分解后的子图只有1条流入（订单）和1条流出（确认单）——丢了“客户信息”流。



修复：子图中必须有1.1或1.2节点接收"客户信息"流，与父图保持外部数据流一致。

恍然大悟：DFD分层为什么必要？因为单张图画不下复杂系统——画太细会“密密麻麻看不清”，画太粗会“什么细节都没有”。分层是软件工程对抗复杂度的通用武器：自顶向下分而治之，每层只关心当前抽象层次的问题。父子平衡原则的本质是“分而治之但不能丢东西”——子图是父图的细化，不是改写。如果子图多/少了数据流，说明分解时引入了新需求或漏掉了原需求，违反“细化不改变本质”的原则。同样的思想在第四章详细设计、第五章测试用例分层都会再出现。

易混辨析

考法	易错点
"DFD的四种符号"	源点/终点是正方形，不是圆形；处理是圆角矩形，不是方形；数据存储是开口矩形，不是箭头
"父图子图平衡"	是子图的外部数据流 = 父图对应处理的数据流，不是"子图处理数量等于父图"
"DFD描述什么"	描述数据流和变换，是逻辑模型，不是物理模型（物理模型用系统流程图）

记忆技巧

四符号口诀："方源圆处开存箭流"——方（正方形）=源点，圆（圆角矩形）=处理，开（开口矩形）=存储，箭（箭头）=数据流。

分层口诀："顶零一二逐层分，父子平衡要对齐"——从顶层逐层往下分，父子图的数据流要平衡。

五、数据字典（DD）

核心概念

数据字典（DD，Data Dictionary）是关于数据的信息的集合，对数据流图中包含的所有元素的定义的集合。

四类元素（必背）

数据字典定义DFD中的四类元素：

- 1. 数据流（数据流图中的箭头）
- 2. 数据项（数据的最小单位）
- 3. 数据存储（数据流图中的开口矩形）
- 4. 处理（数据流图中的圆角矩形）

数据字典的符号表示法（必考，画图/书写题）

数据字典用一套符号来精确描述数据结构，类似正则表达式的"数据语法"：

符号	含义	例子
=	由...组成/定义为	订票单 = 姓名 + 日期 + 航班号
+	与/连接	姓名 + 日期（姓名和日期都有）
{ }	重复	{座位号}（可重复多个座位号）
[]	或/选择	[现金\
0	可选	(备注)（备注可有可无）

完整例子：

- 订票单 = 姓名 + 日期 + 航班号 + {座位号} + [现金|信用卡] + (备注)
- 含义：订票单由姓名、日期、航班号组成，可重复多个座位号，支付方式选现金或信用卡，备注可选

恍然大悟：数据字典为什么用这些符号？因为它是"数据的形式化语法"——用=定义、用+连接、用{ }表重复、用[]表选择、用0表可选，就像写正则表达式一样精确描述数据结构。DFD只画了"有什么数据

流", 但没说"这个数据流里具体包含什么", 数据字典用符号补上了这个精确定义。

数据字典与DFD的关系（高频考点）

DFD和数据字典互为补充，共同构成系统的规格说明（逻辑模型）：

- DFD给出了直观的图形，但没有精确的定义
- 数据字典给出了精确的定义，但没有图形
- 没有数据字典，DFD就不严格；没有DFD，数据字典也难发挥作用
- 两者结合才能完整描述系统

易混辨析

考法	易错点
"数据字典和DFD的关系"	是互补关系，不是替代关系！两者结合才构成完整规格说明
"数据字典定义什么"	定义DFD的四类元素（数据流、数据项、数据存储、处理），不是定义"数据类型"或"变量"

记忆技巧

口诀："流项存处"（数据流、数据项、数据存储、处理）——DFD四种符号里除了"源点/终点"（那是外部实体，不需要定义），其余三个加上"数据项"就是数据字典的四类元素。

关系口诀："图给形，典给义，图文合一才完整"——DFD给图形，数据字典给定义，合起来才完整。

六、成本效益分析

核心概念

成本效益分析是经济可行性研究的核心内容，通过比较开发成本和预期收益来判断项目经济上是否可行。

关键概念

1. 货币的时间价值：同样一笔钱，现在的价值高于未来的价值（因为可以投资生息）
2. 贴现/现值：把未来的钱折算成现在的价值
3. 投资回收期：收回全部投资所需的时间
4. 纯收入：整个生命周期内总收益减去总成本

计算公式和完整例子（计算题必考）

现值公式： $PV = FV / (1+i)^n$

- PV = 现值（现在的价值）
- FV = 未来值（未来某年的收益）
- i = 利率
- n = 年数

完整例子：投资10万开发系统，预计3年每年收益4万，利率5%

年份	未来收益(FV)	现值 $PV=FV/(1.05)^n$	累计现值
第1年	4万	$4/(1.05)^1 = 3.81$ 万	3.81万
第2年	4万	$4/(1.05)^2 = 3.63$ 万	7.44万
第3年	4万	$4/(1.05)^3 = 3.46$ 万	10.90万

分析：

- 投资回收期：第2年累计7.44万<10万，第3年累计10.90万>10万 → 约2.7年回本
- 纯收入：总现值10.90万 - 投资10万 = 0.90万 > 0 → 经济可行

恍然大悟：为什么要贴现？因为"现在的1块钱比未来的1块钱值钱"——这1块钱存银行能生息，所以未来的钱要打折成现值才能和现在的成本比。如果不贴现，3年收益12万看起来赚2万；但贴现后只有10.9万，实际只赚0.9万。贴现揭示了"未来的钱没那么值钱"的真相。

记忆技巧

口诀："现在的钱比将来的钱值钱"——这是货币时间价值的核心。所以算收益时要把未来收益"打折"成现值（贴现）。

七、成本估算（三类方法）

核心概念

成本估算是可行性研究和项目计划的重要环节，有三类主要方法：

方法	含义	特点
代码行估算（LOC）	估算源代码行数，乘以每行成本	简单直接，但行数难估准
功能点估算（FP）	根据功能点数估算，考虑输入/输出/查询/文件/接口	不依赖语言，更客观
COCOMO模型	构造性成本模型，用数学公式估算	最精确，分三个层次

三类方法深度拆解

1. 代码行估算（LOC，Lines of Code）

• 是什么：估算项目总代码行数（KLOC=千行代码），乘以单位成本（每行代码的人天/钱），得到总成本

• 怎么估：

1. 找已完成的类似项目作参考（如以前做过相似规模的电商系统8万行）
2. 根据本项目复杂度调整（功能多20% → 估10万行）
3. 用三点估计：最乐观a、最可能m、最悲观b → 期望值 = $(a+4m+b)/6$
4. 总成本 = KLOC × 每KLOC成本（如每KLOC=10人月）

• 优点：直观简单、容易计算、行业有参考数据

• 缺点：

1. 受编程语言影响大——同样功能Python可能5000行，Java要1.5万行，估算偏差大
2. 早期估不准——需求阶段哪知道最终多少行？
3. 不鼓励简洁代码——按行算钱反而鼓励写冗长代码

• 适用阶段：编码后期（已知部分代码量）或类似项目复用估算

2. 功能点估算（FP，Function Point）

• 是什么：根据软件提供的"功能点数"估算成本，与编程语言无关

• 5个功能点要素（必背）：

1. 外部输入（EI）：用户输入数据（如登录表单、添加订单）



2. 外部输出 (EO) : 系统输出 (如打印报表、发送邮件)
 3. 外部查询 (EQ) : 用户查询不修改数据 (如查订单详情)
 4. 内部逻辑文件 (ILF) : 系统内部维护的数据文件 (如用户表、订单表)
 5. 外部接口文件 (EIF) : 与其他系统共享的文件 (如调用第三方支付)
 - 计算步骤:
1. 数出每类要素的数量
 2. 按复杂度 (简单/中等/复杂) 加权 (如简单输入×3、中等×4、复杂×6)
 3. 求和得到未调整功能点UFP
 4. 乘以技术调整因子TCF (考虑性能、可移植性等14个因素) → $FP = UFP \times TCF$
 5. 总成本 = $FP \times \text{单位成本}$
 - 优点:

1. 与语言无关——同样功能Python和Java算出的FP一样
2. 早期可估——需求分析阶段就能数出输入输出
3. 客观可重复——不同人估算结果接近
 - 缺点: 5要素的复杂度判断仍有主观性、TCF因子需要经验、培训成本高
 - 适用阶段: 需求分析阶段 (最常用)

3. COCOMO模型 (Constructive Cost Model)

- 是什么: Barry Boehm 提出的构造性成本模型, 用数学公式从代码规模估算工作量
- 基本公式: $E = a \times (KLOC)^b$
- E = 工作量 (人月)
- $KLOC$ = 千行代码
- a 、 b = 模型参数 (按项目类型查表)
- 三个项目类型 (基本COCOMO的 a/b 值) :
 - 组织型 (Organic) : 小型简单项目 (如小型ERP、工资系统、企业内部管理系统), $a=2.4, b=1.05$
 - 半独立型 (Semidetached) : 中等复杂项目 (如中型电商、银行核心业务、数据库管理系统), $a=3.0, b=1.12$
 - 嵌入式 (Embedded) : 大型复杂项目 (如航天控制软件、嵌入式OS、空管系统), $a=3.6, b=1.20$
- 三层精度递增:

层次	精度	用途	特点
基本COCOMO	最粗略	早期粗略估算	只看KLOC, 按项目类型套公式
中级COCOMO	中等	中期估算	加15个成本驱动因子调整, 公式: $E = a \times KLOC^b \times EAF$ (EAF为15因子相乘)
详细COCOMO	最精确	后期精确估算	把项目分阶段 (需求/设计/编码/测试), 每阶段单独算

- 完整例子: 组织型项目, 估计32 KLOC
- 基本COCOMO: $E = 2.4 \times 32^{1.05} \approx 2.4 \times 38.05 \approx 91$ 人月
- 即需要91个人月工作量, 按15万/人月算总成本约1365万
- 优点: 有数学模型支撑、有大量历史数据校准、考虑因素全面
- 缺点:



1. 依赖KLOC估算——KLOC本身就难估准，COCOMO跟着不准

2. 15个驱动因子主观性大——团队经验"高/中/低"凭谁判断

3. 不适合敏捷项目——敏捷无明确总规模

• 适用阶段：传统瀑布/RUP项目，需求相对明确后

恍然大悟：三种方法不是替代关系，而是互补关系+演进关系。FP在早期（需求阶段）可估、与语言无关；估出FP后用经验值"FP→KLOC"换算（如1FP≈53行Java代码）；得到KLOC后套COCOMO算工作量；项目后期回头用LOC校验估算精度。COCOMO本质是LOC估算的进化版——把"每行多少钱"升级为带工程经验系数的数学模型。三者形成估算流水线：FP（数功能）→ KLOC（估规模）→ COCOMO（算工作量）。考试常考的"LOC vs FP"区别要记牢：LOC受语言影响，FP不受。

易混辨析

考法	易错点
"COCOMO三层精度"	基本→中级→详细，精度递增。不要记反
"LOC vs FP"	LOC按代码行，受编程语言影响；FP按功能点，不受语言影响
"COCOMO公式"	$E = a \times (KLOC)^b$ (KLOC)^b, b是指数 (>1表示规模越大，工作量增长越快)
"FP的5要素"	输入、输出、查询、内部文件、外部接口——不要漏"外部接口"

记忆技巧

COCOMO口诀："基中详，越来越准"——基本、中级、详细，精度越来越高。

FP五要素口诀："输入输出查询，内部外部文件"——5个要素覆盖了系统的所有外部交互。

LOC vs FP："LOC看皮（代码外表），FP看肉（功能本质）"——LOC受皮（语言）影响，FP直击肉（功能）。

考点聚焦

高频考点清单

1. DFD四种符号的图形对应
2. 父图与子图的平衡原则
3. 三类可行性的场景判断（尤其技术vs操作）
4. 数据字典与DFD的互补关系
5. 数据字典四类元素
6. COCOMO三层精度
7. 可行性研究的实质
8. 系统流程图vs DFD（物理vs逻辑）

常见题型

- 符号对应：DFD四种符号分别是什么形状
- 场景判断：给定项目情况判断属于哪类可行性



- 关系判断：数据字典与DFD的关系（互补，非替代）
 - 平衡原则：判断DFD分层是否正确
-

记忆口诀总览

1. 可行性实质："小代价、短时间、定值不值"
 2. 三类可行性："技经操（机灵操）"
 3. DFD四符号："方源圆处开存箭流"
 4. DFD分层："顶零一二逐层分，父子平衡要对齐"
 5. 数据字典四元素："流项存处"
 6. 图典关系："图给形，典给义，图文合一才完整"
 7. COCOMO："基中详，越来越准"
 8. 系统流程图vs DFD："系流物理，DFD逻辑"
-

本章小结

本章核心逻辑链：问题值不值得解决（可行性研究）→ 从技术/经济/操作三维度评估 → 用DFD和数据字典建立逻辑模型 → 用成本效益分析算经济账 → 用COCOMO等方法估算成本。

本章最重要的两个工具是DFD和数据字典——它们不仅用于可行性研究，也是下一章需求分析的核心工具。掌握DFD的四种符号、分层原则、父子图平衡，以及数据字典的四类元素和与DFD的互补关系，是本章拿分的关键。可行性三类维度的场景判断是选择题的高频设坑点，务必分清"技术（能不能实现）"和"操作（能不能用起来）"。

